ARL Debugging Challenge: Rover Safety Monitor

Debug Report

Name: Omar Ahmed Abdel Hameed

Challenge: ARL 2027 Workshop — Rover Safety Monitor Debugging Challenge

---

1. System Overview

The rover safety monitor processes raw obstacle detections from a radar sensor. Each detection is in the rover's **local coordinate frame** (forward/left distances). The system must:

1. Filter out invalid detections based on confidence, range, and data validity.
2. Convert valid detections from local rover coordinates into **world coordinates** using the rover's pose (position + heading).
3. Identify the **nearest obstacle**.
4. Decide whether **emergency braking** is required based on the obstacle's position relative to the lane corridor and stopping distance.

---

2. Bugs Found and Fixed

Bug 1 — Heading Not Converted to Radians

File: `safety_monitor.cpp`

Symptom: World coordinate calculations were completely wrong, producing garbage X/Y values for all obstacles.

Root Cause: The heading was passed directly to `std::cos()` and `std::sin()` in degrees, but these functions expect radians.

```
// BEFORE (wrong)
const double headingRadians = pose.headingDegrees;

// AFTER (correct)
const double headingRadians = pose.headingDegrees * (PI / 180.0);
```

Concept: Trigonometric functions in C++ (and most languages) operate in radians. Degrees must be converted using the formula: `radians = degrees × π / 180`.

Bug 2 — Loop Skips the Last Detection (Wait, actually modern C++ fix applied)

File: `safety_monitor.cpp`

Symptom: The last detection in the list was never processed.

Root Cause: The original loop used `for (std::size_t index = 0; index + 1 < detections.size(); ++index)`, skipping the last element.

Fix: Changed to a range-based for loop for safety and clarity.

```
// BEFORE (wrong — skips last element)
for (std::size_t index = 0; index + 1 < detections.size(); ++index) {
    const auto & detection = detections[index];
    // ...
}

// AFTER (correct - range-based)
for (const auto & detection : detections) {
    // ...
}
```

Concept: Off-by-one error. Using range-based for loops avoids this class of error entirely.

Bug 3 — Confidence Check Logic Inverted

File: `safety_monitor.cpp`

Symptom: Detections with low confidence were accepted, while high-confidence detections were rejected — the opposite of intended behaviour.

Root Cause: The condition checked that confidence was *at or below* the minimum threshold, instead of *at or above* it.

```
// BEFORE (wrong — accepts low confidence)
const bool validConfidence = detection.confidence >= 0.0
    && detection.confidence <= config.minimumConfidence;

// AFTER (correct — accepts confidence >= minimum, up to 1.0)
const bool validConfidence = detection.confidence >= 0.0
    && detection.confidence >= config.minimumConfidence && detection.co
nfidence <= 1.0;
```

Concept: The minimum confidence is a lower bound — valid detections must meet or exceed it. The original condition treated it as an upper bound, inverting the filter entirely.

Bug 4 — `findNearestObstacle` Returns the Farthest Obstacle

File: `safety_monitor.cpp`

Symptom: The reported "nearest" obstacle was actually the one furthest away.

Root Cause: The comparison operator was `>` instead of `<`, so the loop kept replacing the candidate whenever it found something farther, not closer.

```
// BEFORE (wrong — finds farthest)
if (obstacle.range > nearest->range)

// AFTER (correct — finds nearest)
if (obstacle.range < nearest->range)
```

Concept: Finding a minimum value in a list requires replacing the current best when a *smaller* value is found, not a larger one.

Bug 5 — `shouldEmergencyBrake` Always Returns `false`

File: `safety_monitor.cpp`

Symptom: Emergency braking never engaged regardless of obstacle proximity, lane position, or speed.

Root Cause: The function body was a hardcoded `return false;` with no actual logic implemented.

Fix: Implemented the full braking decision logic per the specification — an obstacle triggers braking if all three conditions hold: - It is in front of the rover (`forward >= 0.0`) - It is within the lane corridor (`fabs(left) <= laneHalfWidth`) - Its forward distance does not exceed the stopping distance (`forward <= stoppingDistance`)

```
// AFTER (correct)
bool shouldEmergencyBrake(const std::vector<Obstacle>& obstacles, doubl
e speedKph, const SafetyConfig& config) {
    const auto stoppingDist = calculateStoppingDistance(speedKph,confi
g);
    for (const auto &obstacle: obstacles) {
        if (obstacle.forward >= 0.0 &&
            fabs(obstacle.left) <= config.laneHalfWidthMeters &&
            obstacle.forward <= stoppingDist) return true;

    }
    return false;
}
```

Concept: A stub function returning a hardcoded value is a common placeholder during development. It must be replaced with real logic before the system is considered functional.

Bug 6 — Speed Not Converted from kph to m/s in `calculateStoppingDistance`

File: `safety_monitor.cpp`

Symptom: Stopping distance was calculated using the raw kph value instead of m/s, producing a result roughly 3.6× too large.

Root Cause: The conversion line was present but the actual division by 3.6 was missing (or was just a placeholder).

```
// BEFORE (wrong)
const double speedMps = speedKph;

// AFTER (correct)
const double speedMps = speedKph / 3.6;
```

Concept: Physics formulas for reaction distance and braking distance require speed in metres per second. 1 kph = 1/3.6 m/s.

Bug 7 — Incorrect Coordinate Transformation (Wrong sign for Sine)

File: `safety_monitor.cpp`

Symptom: Obstacles were projected to incorrect world coordinates, especially when the rover had a non-zero heading. Objects on the left were appearing on the wrong side in the world frame.

Root Cause: The 2D rotation matrix for world coordinates was implemented with a `+` instead of a `-` for the sine term in the X-coordinate calculation.

```
// BEFORE (wrong)
pose.worldX + cosine * detection.forward + sine * detection.left,

// AFTER (correct)
pose.worldX + cosine * detection.forward - sine * detection.left,
```

Concept: The standard 2D rotation for a point (x, y) by angle θ is $x' = x\cos\theta - y\sin\theta$ and $y' = x\sin\theta + y\cos\theta$. In this case, `forward` is x and `left` is y.

---

3. How a Detection Flows Through the Corrected System

Using the **cone** detection as an example: - Input: `{ "cone", forward=12.0, left=4.0, confidence=0.95 }` - Rover pose: `worldX=100, worldY=50,` heading=90° - Config: `minConfidence=0.60, maxRange=40.0, laneHalfWidth=1.50`

Step 1 — Validation: - All fields are finite ✓ - Confidence: `0.95 >= 0.60` ✓ - Range: `hypot(12.0, 4.0) ≈ 12.65`, which is `> 0` and `<= 40.0` ✓ - Detection is valid.

Step 2 — Coordinate Conversion: - Heading 90° → π/2 radians → `cos = 0, sin = 1` - The rotation formula applied is: - `worldX = poseX + cosθ × forward - sinθ × left` → `100 + 0×12 - 1×4 = 96.0` - `worldY = poseY + sinθ × forward + cosθ × left` → `50 + 1×12 + 0×4 = 62.0`

Step 3 — Nearest Obstacle: - Cone has range ≈ 12.65m, which is the smallest among valid detections → cone is nearest.

Step 4 — Emergency Braking: - Speed: `36 kph = 10 m/s` - Stopping distance: `(10 × 1.0) + (10² / (2 × 5.0)) = 10 + 10 = 20.0m` - Cone: `forward=12.0 >= 0` ✓, `fabs(left=4.0) <= 1.50` ✗ — cone is outside the lane corridor. - Barrier: `forward=18.0`, `fabs(left=1.0) <= 1.50` ✓, `18.0 <= 20.0` ✓, `forward >= 0` ✓ → **ENGAGE**

---

4. References and Resources

- C++ `<cmath>` documentation — `std::cos`, `std::sin`, `std::hypot`, `std::isfinite`, `std::fabs`
- 2D rotation matrix derivation (standard linear algebra)
- ARL 2027 Workshop problem specification (README)
- AI assistance (Claude by Anthropic) — used for conceptual explanation and reasoning guidance. All fixes were understood and implemented independently.

---

5. Proof of Operation

The following image demonstrates the system correctly processing detections, transforming coordinates, and identifying the nearest obstacle requiring action: